

In previous chapters we discussed how to set up models. Inference then corresponds to operations such as summing over subsets of variables. In machine learning and related areas we will often deal with distributions containing hundreds of variables. In general inference is computationally very expensive and it is useful to understand for which graphical structures this could be cheap in order that we may make models which we can subsequently compute with. In this chapter we discuss inference in a cheap case, namely trees, which has links to classical algorithms in many different fields from computer science (dynamic programming) to physics (transfer matrix methods).

5.1 Marginal Inference

Given a distribution $p(x_1, \dots, x_n)$, *inference* is the process of computing functions of the distribution. *Marginal inference* is concerned with the computation of the distribution of a subset of variables, possibly conditioned on another subset. For example, given a joint distribution $p(x_1, x_2, x_3, x_4, x_5)$ and evidence $x_1 = \text{tr}$, a marginal inference calculation is

$$p(x_5 | x_1 = \text{tr}) \propto \sum_{x_2, x_3, x_4} p(x_1 = \text{tr}, x_2, x_3, x_4, x_5). \quad (5.1.1)$$

Marginal inference for discrete models involves summation and will be the focus of our development. In principle the algorithms carry over to continuous variable models although the lack of closure of most continuous distributions under marginalisation (the Gaussian being a notable exception) can make the direct transference of these algorithms to the continuous domain problematic. The focus here is on efficient inference algorithms for marginal inference in singly connected structures. An efficient algorithm for multiply connected graphs will be considered in chapter(6).

5.1.1 Variable elimination in a Markov chain and message passing

A key concept in efficient inference is *message passing* in which information from the graph is summarised by local edge information. To develop this idea, consider the four variable Markov chain (Markov chains are

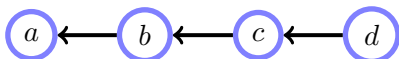


Figure 5.1: A Markov chain is of the form $p(x_T) \prod_{t=1}^{T-1} p(x_t | x_{t+1})$ for some assignment of the variables to labels x_t . Variable Elimination can be carried out in time linear in the number of variables in the chain.

discussed in more depth in section(23.1))

$$p(a, b, c, d) = p(a|b)p(b|c)p(c|d)p(d) \quad (5.1.2)$$

as given in fig(5.1), for which our task is to calculate the marginal $p(a)$. For simplicity, we assume that each of the variables has domain $\{0, 1\}$. Then

$$p(a = 0) = \sum_{b \in \{0,1\}, c \in \{0,1\}, d \in \{0,1\}} p(a = 0, b, c, d) = \sum_{b \in \{0,1\}, c \in \{0,1\}, d \in \{0,1\}} p(a = 0|b)p(b|c)p(c|d)p(d) \quad (5.1.3)$$

We could carry out this computation by simply summing each of the probabilities for the $2 \times 2 \times 2 = 8$ states of the variables b, c and d . This would therefore require 7 addition-of-two-numbers calls.

A more efficient approach is to push the summation over d as far to the right as possible:

$$p(a = 0) = \sum_{b \in \{0,1\}, c \in \{0,1\}} p(a = 0|b)p(b|c) \underbrace{\sum_{d \in \{0,1\}} p(c|d)p(d)}_{\gamma_d(c)} \quad (5.1.4)$$

where $\gamma_d(c)$ is a (two state) potential. Defining $\gamma_d(c)$ requires two addition-of-two-numbers calls, one call for each state of c . Similarly, we can distribute the summation over c as far to the right as possible:

$$p(a = 0) = \sum_{b \in \{0,1\}} p(a = 0|b) \underbrace{\sum_{c \in \{0,1\}} p(b|c)\gamma_d(c)}_{\gamma_c(b)} \quad (5.1.5)$$

Then, finally,

$$p(a = 0) = \sum_{b \in \{0,1\}} p(a = 0|b)\gamma_c(b) \quad (5.1.6)$$

By distributing the summations we have made $3 \times 2 - 1 = 5$ addition-of-two-numbers calls, compared to $2^3 - 1 = 7$ from the naive approach. Whilst this saving may not appear much, the important point is that the number of computations for a chain of length $T + 1$ would be linear, $2T$, as opposed to exponential, $2^T - 1$ for the naive approach. @@

This procedure is called *variable elimination* since each time we sum over the states of a variable we eliminate it from the distribution. We can always perform variable elimination in a chain efficiently since there is a natural way to distribute the summations, working inwards from the edges. Note that in the above case, the potentials are in fact always distributions – we are just recursively computing the marginal distribution of the right leaf of the chain.

One can view the elimination of a variable as passing a *message* (information) to a neighbouring node on the graph. We can calculate a univariate-marginal of any tree (singly connected graph) by starting at a leaf of the tree, eliminating the variable there, and then working inwards, nibbling off each time a leaf of the remaining tree. Provided we perform elimination from the leaves inwards, then the structure of the remaining graph is simply a subtree of the original tree, albeit with the conditional probability table entries modified. This is guaranteed to enable us to calculate any marginal $p(x_i)$ using a number of summations which scales linearly with the number of variables in the tree.

Finding conditional marginals for a chain

Consider the following inference problem, fig(5.1) : Given

$$p(a, b, c, d) = p(a|b)p(b|c)p(c|d)p(d), \quad (5.1.7)$$

find $p(d|a)$. This can be computed using

$$p(d|a) \propto \sum_{b,c} p(a, b, c, d) = \sum_{b,c} p(a|b)p(b|c)p(c|d)p(d) = \sum_c \underbrace{\sum_b p(a|b)p(b|c)p(c|d)p(d)}_{\gamma_b(c)} \equiv \gamma_c(d) \quad (5.1.8)$$

The missing proportionality constant is found by repeating the computation for all states of variable d . Since we know that $p(d|\mathbf{a}) = k\gamma_c(d)$, where $\gamma_c(d)$ is the unnormalised result of the summation, we can use the fact that $\sum_d p(d|\mathbf{a}) = 1$ to infer that $k = 1/\sum_d \gamma_c(d)$.

In this example, the potential $\gamma_b(c)$ is not a distribution in c , nor is $\gamma_c(d)$. In general, one may view variable elimination as the passing of messages in the form of potentials from nodes to their neighbours. For belief networks variable elimination passes messages that are distributions when following the direction of the edge, and non-normalised potentials when passing messages against the direction of the edge.

Remark 5.1 (Variable elimination in trees as matrix multiplication). Variable elimination is related to the associativity of matrix multiplication. For equation (5.1.2) above, we can define matrices

$$\begin{aligned} [\mathbf{M}_{ab}]_{i,j} &= p(a=i|b=j), \quad [\mathbf{M}_{bc}]_{i,j} = p(b=i|c=j), \\ [\mathbf{M}_{cd}]_{i,j} &= p(c=i|d=j), \quad [\mathbf{M}_d]_i = p(d=i), \quad [\mathbf{M}_a]_i = p(a=i) \end{aligned} \quad (5.1.9)$$

Then the marginal $\mathbf{M}_a$ can be written

$$\mathbf{M}_a = \mathbf{M}_{ab}\mathbf{M}_{bc}\mathbf{M}_{cd}\mathbf{M}_d = \mathbf{M}_{ab}(\mathbf{M}_{bc}(\mathbf{M}_{cd}\mathbf{M}_d)) \quad (5.1.10)$$

since matrix multiplication is associative. This matrix formulation of calculating marginals is called the *transfer matrix* method, and is particularly popular in the physics literature[27].

Example 5.1 (Where will the fly be?).

You live in a house with three rooms, labelled 1, 2, 3. There is a door between rooms 1 and 2 and another between rooms 2 and 3. One cannot directly pass between rooms 1 and 3 in one time-step. An annoying fly is buzzing from one room to another and there is some smelly cheese in room 1 which seems to attract the fly more. Using x_t to indicate which room the fly is in at time t , with $\text{dom}(x_t) = \{1, 2, 3\}$, the movement of the fly can be described by a transition

$$p(x_{t+1}=i|x_t=j) = M_{ij} \quad (5.1.11)$$

where M_{ij} is an element of the transition matrix

$$\mathbf{M} = \begin{pmatrix} 0.7 & 0.5 & 0 \\ 0.3 & 0.3 & 0.5 \\ 0 & 0.2 & 0.5 \end{pmatrix} \quad (5.1.12)$$

The matrix $\mathbf{M}$ is called ‘stochastic’ meaning that, as required of a conditional probability table, its columns sum to 1, $\sum_{i=1}^3 M_{ij} = 1$. Given that the fly is in room 1 at time $t = 1$, what is the probability of room occupancy at time $t = 5$? Assume a Markov chain which is defined by the joint distribution

$$p(x_1, \dots, x_T) = p(x_1) \prod_{t=1}^{T-1} p(x_{t+1}|x_t) \quad (5.1.13)$$

We are asked to compute $p(x_5|x_1=1)$ which is given by

$$\sum_{x_4, x_3, x_2} p(x_5|x_4)p(x_4|x_3)p(x_3|x_2)p(x_2|x_1=1) \quad (5.1.14)$$

Since the graph of the distribution is a Markov chain, we can easily distribute the summation over the terms. This is most easily done using the transfer matrix method, giving

$$p(x_5=i|x_1=1) = [\mathbf{M}^4 \mathbf{v}]_i \quad (5.1.15)$$

where $\mathbf{v}$ is a vector with components $(1, 0, 0)^\top$, reflecting the evidence that at time $t = 1$ the fly is in room 1. Computing this we have (to 4 decimal places of accuracy)

$$\mathbf{M}^4 \mathbf{v} = \begin{pmatrix} 0.5746 \\ 0.3180 \\ 0.1074 \end{pmatrix} \quad (5.1.16)$$

Similarly, at time $t = 6$, the occupancy probabilities are $(0.5612, 0.3215, 0.1173)$. The room occupancy probability is converging to a particular distribution – the *stationary* distribution of the Markov chain. One might ask where the fly is after an *infinite* number of time-steps. That is, we are interested in the large t behaviour of

$$p(x_{t+1}) = \sum_{x_t} p(x_{t+1}|x_t)p(x_t) \quad (5.1.17)$$

At convergence $p(x_{t+1}) = p(x_t)$. Writing $\mathbf{p}$ for the vector describing the stationary distribution, this means

$$\mathbf{p} = \mathbf{M}\mathbf{p} \quad (5.1.18)$$

In other words, $\mathbf{p}$ is the eigenvector of $\mathbf{M}$ with eigenvalue 1 [135]. Computing this numerically, the stationary distribution is $(0.5435, 0.3261, 0.1304)$. Note that software packages usually return eigenvectors with $\sum_i e_i^2 = 1$ — the unit eigenvector therefore will usually require normalisation to make this a probability with $\sum_i e_i = 1$.

5.1.2 The sum-product algorithm on factor graphs

Both Markov and belief networks can be represented using factor graphs. For this reason it is convenient to derive a marginal inference algorithm for FGs since this then applies to both Markov and belief networks. This is termed the sum-product algorithm since to compute marginals we need to distribute the sum over variable states over the product of factors. In other texts, this is also referred to as *belief propagation*.

Non-branching graphs : variable to variable messages

Consider the distribution

$$p(a, b, c, d) = f_1(a, b) f_2(b, c) f_3(c, d) f_4(d) \quad (5.1.19)$$

which has the factor graph represented in fig(5.2). To compute the marginal $p(a, b, c)$, since the variable d only occurs locally, we use

$$p(a, b, c) = \sum_d p(a, b, c, d) = \sum_d f_1(a, b) f_2(b, c) f_3(c, d) f_4(d) = f_1(a, b) f_2(b, c) \underbrace{\sum_d f_3(c, d) f_4(d)}_{\mu_{d \rightarrow c}(c)} \quad (5.1.20)$$

Here $\mu_{d \rightarrow c}(c)$ defines a message from node d to node c and is a function of the variable c . Similarly,

$$p(a, b) = \sum_c p(a, b, c) = f_1(a, b) \underbrace{\sum_c f_2(b, c) \mu_{d \rightarrow c}(c)}_{\mu_{c \rightarrow b}(b)} \quad (5.1.21)$$

Hence

$$\mu_{c \rightarrow b}(b) = \sum_c f_2(b, c) \mu_{d \rightarrow c}(c) \quad (5.1.22)$$

It is clear how one can recurse this definition of messages so that for a chain of n variables the marginal of the first node can be computed in time linear in n . The term $\mu_{c \rightarrow b}(b)$ can be interpreted as carrying

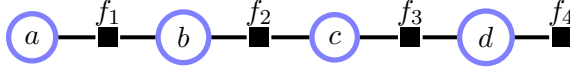


Figure 5.2: For singly connected structures without branches, simple messages from one variable to its neighbour may be defined to form an efficient marginal inference scheme.

marginal information from the graph beyond c . For simple linear structures with no branching, messages from variables to variables are sufficient. However, as we will see below in more general structures with branching, it is useful to consider two types of messages, namely those from variables to factors and vice versa.

General singly connected factor graphs

The slightly more complex example,

$$p(a|b)p(b|c,d)p(c)p(d)p(e|d) \quad (5.1.23)$$

has the factor graph depicted in fig(5.3)

$$f_1(a, b) f_2(b, c, d) f_3(c) f_4(d, e) f_5(d) \quad (5.1.24)$$

The marginal $p(a, b)$ can be represented by an amputated graph with a message, since

$$p(a, b) = f_1(a, b) \underbrace{\sum_{c,d} f_2(b, c, d) f_3(c) f_5(d) \sum_e f_4(d, e)}_{\mu_{f_2 \rightarrow b}(b)} \quad (5.1.25)$$

where $\mu_{f_2 \rightarrow b}(b)$ is a message from a factor to a variable. This message can be constructed from messages arriving from the two branches through c and d , namely

$$\mu_{f_2 \rightarrow b}(b) = \sum_{c,d} f_2(b, c, d) \underbrace{f_3(c)}_{\mu_{c \rightarrow f_2}(c)} \underbrace{f_5(d) \sum_e f_4(d, e)}_{\mu_{d \rightarrow f_2}(d)} \quad (5.1.26)$$

Similarly, we can interpret

$$\mu_{d \rightarrow f_2}(d) = \underbrace{f_5(d)}_{\mu_{f_5 \rightarrow d}(d)} \underbrace{\sum_e f_4(d, e)}_{\mu_{f_4 \rightarrow d}(d)} \quad (5.1.27)$$

To complete the interpretation we identify $\mu_{c \rightarrow f_2}(c) \equiv \mu_{f_3 \rightarrow c}(c)$. In a non-branching link, one can more simply use a variable to variable message. To compute the marginal $p(a)$, we then have

$$p(a) = \underbrace{\sum_b f_1(a, b) \mu_{f_2 \rightarrow b}(b)}_{\mu_{f_1 \rightarrow a}(a)} \quad (5.1.28)$$

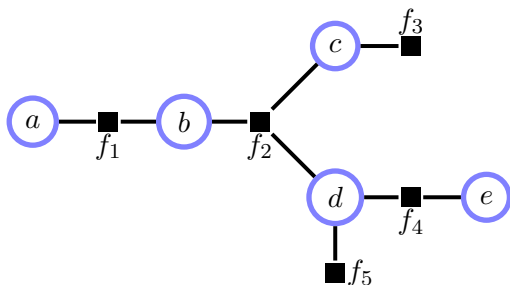


Figure 5.3: For a branching singly connected graph, it is useful to define messages from both factors to variables, and variables to factors.

For consistency of interpretation, one also can view the above as

$$\mu_{f_1 \rightarrow a}(a) = \sum_b f_1(a, b) \underbrace{\mu_{f_2 \rightarrow b}(b)}_{\mu_{b \rightarrow f_1}(b)} \quad (5.1.29)$$

We can now see how a message from a factor to a node is formed from summing the product of incoming node-to-factor messages. Similarly, a message from a node to a factor is given by the product of incoming factor-to-node messages.

A convenience of this approach is that the messages can be reused to evaluate other marginal inferences. For example, it is clear that $p(b)$ is given by

$$p(b) = \sum_a \underbrace{f_1(a, b)}_{\mu_{f_1 \rightarrow b}(b)} \mu_{f_2 \rightarrow b}(b) \quad (5.1.30)$$

If we additionally desire $p(c)$, we need to define the message from f_2 to c ,

$$\mu_{f_2 \rightarrow c}(c) = \sum_{b,d} f_2(b, c, d) \mu_{b \rightarrow f_2}(b) \mu_{d \rightarrow f_2}(d) \quad (5.1.31)$$

where $\mu_{b \rightarrow f_2}(b) \equiv \mu_{f_1 \rightarrow b}(b)$. This demonstrates the reuse of already computed message from d to f_2 to compute the marginal $p(c)$.

Definition 5.1 (Message schedule). A message schedule is a specified sequence of message updates. A valid schedule is that a message can be sent from a node only when that node has received all requisite messages from its neighbours. In general, there is more than one valid updating schedule.

Sum-Product algorithm

The sum-product algorithm is described below in which messages are updated as a function of incoming messages. One then proceeds by computing the messages in a schedule that allows the computation of a new message based on previously computed messages, until all messages from all factors to variables and vice-versa have been computed.

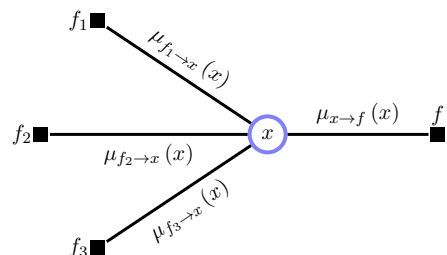
Procedure 5.1 (Sum-Product messages on Factor Graphs).

Given a distribution defined as a product on subsets of the variables, $p(\mathcal{X}) = \frac{1}{Z} \prod_f \phi_f(\mathcal{X}_f)$, provided the factor graph is singly connected we can carry out summation over the variables efficiently.

Initialisation Messages from leaf node factors are initialised to the factor. Messages from leaf variable nodes are set to unity.

Variable to Factor message

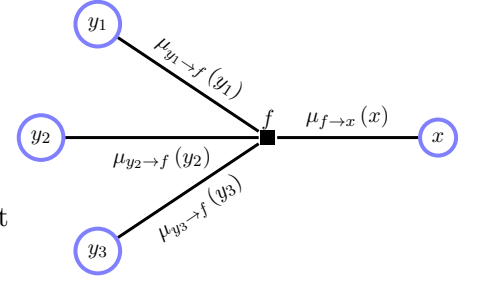
$$\mu_{x \rightarrow f}(x) = \prod_{g \in \{\text{ne}(x) \setminus f\}} \mu_{g \rightarrow x}(x)$$



Factor to Variable message

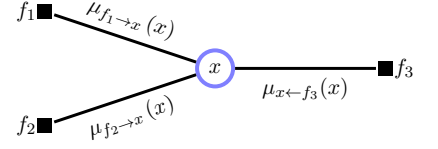
$$\mu_{f \rightarrow x}(x) = \sum_{\mathcal{X}_f \setminus x} \phi_f(\mathcal{X}_f) \prod_{y \in \{\text{ne}(f) \setminus x\}} \mu_{y \rightarrow f}(y)$$

We write $\sum_{\mathcal{X}_f \setminus x}$ to denote summation over all states in the set of variables $\mathcal{X}_f \setminus x$.



Marginal

$$p(x) \propto \prod_{f \in \text{ne}(x)} \mu_{f \rightarrow x}(x)$$



For marginal inference, the important information is the relative size of the message states so that we may renormalise messages as we wish. Since the marginal will be proportional to the incoming messages for that variable, the normalisation constant is trivially obtained using the fact that the marginal must sum to 1. However, if we wish to also compute any normalisation constant using these messages, we cannot normalise the messages since this global information will then be lost.

5.1.3 Dealing with Evidence

For a distribution which splits into evidential and non-evidential variables, $\mathcal{X} = \mathcal{X}_e \cup \mathcal{X}_n$, the marginal of a non-evidential variable $p(x_i, \mathcal{X}_e)$ is given by summing over all the variables in $\mathcal{X}_n$ (except for x_i) with $\mathcal{X}_e$ set into their evidential states. There are two ways to reconcile this with the factor graph formalism. Either we can simply say that by setting the variables $\mathcal{X}_e$, we define a new factor graph on $\mathcal{X}_n$, and then pass messages on this new factor graph. Alternatively, we can define the potentials which contain the variables $\mathcal{X}_e$ by multiplying each potential that contains an evidential variable by a delta function (an indicator) that is zero unless the variable x_e is in the specified evidential state. When we then perform a factor to variable message, the sum of this modified potential over any of the evidential variable states will be zero except for that state which corresponds to the evidential setting. Another way to view this is that the sum in the factor to variable message is only over the non-evidential variables, with any evidential variables in the potential set into their evidential states.

5.1.4 Computing the marginal likelihood

For a distribution defined as products over potentials $\phi_f(\mathcal{X}_f)$

$$p(\mathcal{X}) = \frac{1}{Z} \prod_f \phi_f(\mathcal{X}_f) \tag{5.1.32}$$

the normalisation is given by

$$Z = \sum_{\mathcal{X}} \prod_f \phi_f(\mathcal{X}_f) \tag{5.1.33}$$

To compute this summation efficiently we take the product of all incoming messages to an arbitrarily chosen variable x and then sum over the states of that variable:

$$Z = \sum_x \prod_{f \in \text{ne}(x)} \mu_{f \rightarrow x}(x) \tag{5.1.34}$$

If the factor graph is derived from setting a subset of variables of a BN in evidential states then the summation over all non-evidential variables will yield the marginal on the visible (evidential) variables. For example

$$p(\mathbf{b}, \mathbf{d}) = \sum_{a, c} p(a|\mathbf{b})p(\mathbf{b}|c)p(c|\mathbf{d})p(\mathbf{d}). \tag{5.1.35}$$

This can be interpreted as requiring the sum over a product of suitably defined factors. Hence one can readily find the marginal likelihood of the evidential variables for singly connected BNs.

Log messages

For the above method to work, the absolute (not relative) values of the messages are required, which prohibits renormalisation at each stage of the message passing procedure. However, without normalisation the numerical value of messages can become very small, particularly for large graphs, and numerical precision issues can occur. A remedy in this situation is to work with log messages,

$$\lambda = \log \mu \quad (5.1.36)$$

For this, the variable to factor messages

$$\mu_{x \rightarrow f}(x) = \prod_{g \in \{\text{ne}(x) \setminus f\}} \mu_{g \rightarrow x}(x) \quad (5.1.37)$$

become simply

$$\lambda_{x \rightarrow f}(x) = \sum_{g \in \{\text{ne}(x) \setminus f\}} \lambda_{g \rightarrow x}(x) \quad (5.1.38)$$

More care is required for the factors to variable messages, which are defined by

$$\mu_{f \rightarrow x}(x) = \sum_{\mathcal{X}_f \setminus x} \phi_f(\mathcal{X}_f) \prod_{y \in \{\text{ne}(f) \setminus x\}} \mu_{y \rightarrow f}(y) \quad (5.1.39)$$

Naively, one may write

$$\lambda_{f \rightarrow x}(x) = \log \left(\sum_{\mathcal{X}_f \setminus x} \phi_f(\mathcal{X}_f) \exp \left(\sum_{y \in \{\text{ne}(f) \setminus x\}} \lambda_{y \rightarrow f}(y) \right) \right) \quad (5.1.40)$$

However, the exponentiation of the log messages will cause potential numerical precision problems. A solution to this numerical difficulty is obtained by finding the largest value of the incoming log messages,

$$\lambda_{y \rightarrow f}^* = \max_{y \in \{\text{ne}(f) \setminus x\}} \lambda_{y \rightarrow f}(y) \quad (5.1.41)$$

Then

$$\lambda_{f \rightarrow x}(x) = \lambda_{y \rightarrow f}^* + \log \left(\sum_{\mathcal{X}_f \setminus x} \phi_f(\mathcal{X}_f) \exp \left(\sum_{y \in \{\text{ne}(f) \setminus x\}} \lambda_{y \rightarrow f}(y) - \lambda_{y \rightarrow f}^* \right) \right) \quad (5.1.42)$$

By construction the terms $\exp \left(\sum_{y \in \{\text{ne}(f) \setminus x\}} \lambda_{y \rightarrow f}(y) - \lambda_{y \rightarrow f}^* \right)$ will be ≤ 1 , with at least one term being equal to 1. This ensures that the dominant numerical contributions to the summation are computed accurately.

Log marginals are readily found using

$$\log p(x) = \sum_{f \in \text{ne}(x)} \lambda_{f \rightarrow x}(x) \quad (5.1.43)$$

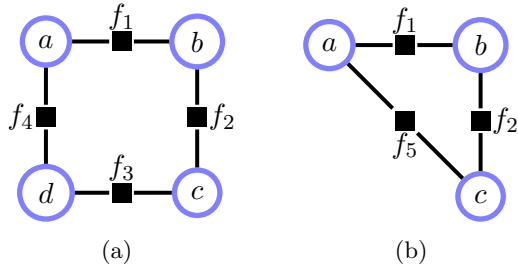


Figure 5.4: (a): Factor graph with a loop. (b): Eliminating the variable d adds an edge between a and c , demonstrating that, in general, one cannot perform marginal inference in loopy graphs by simply passing messages along existing edges in the original graph.

5.1.5 The problem with loops

Loops cause a problem with variable elimination (or message passing) techniques since once a variable is eliminated the structure of the ‘amputated’ graph in general changes. For example, consider the FG

$$p(a, b, c, d) = f_1(a, b) f_2(b, c) f_3(c, d) f_4(a, d) \quad (5.1.44)$$

depicted in fig(5.4a). The marginal $p(a, b, c)$ is given by

$$p(a, b, c) = f_1(a, b) f_2(b, c) \underbrace{\sum_d f_3(c, d) f_4(a, d)}_{f_5(a, c)} \quad (5.1.45)$$

which adds a link ac in the amputated graph, see fig(5.4b). This means that one cannot account for information from variable d by simply updating potentials on links in the original graph – one needs to account for the fact that the structure of the graph changes. The junction tree algorithm, chapter(6) deals with this by combining variables to make a new singly connected graph for which the graph structure remains singly connected under variable elimination.

5.2 Other Forms of Inference

5.2.1 Max-Product

A common interest is the most likely state of distribution. That is

$$\operatorname{argmax}_{x_1, x_2, \dots, x_n} p(x_1, x_2, \dots, x_n) \quad (5.2.1)$$

To compute this efficiently for trees we exploit any factorisation structure of the distribution, analogous to the sum-product algorithm. That is, we aim to distribute the maximization so that only local computations are required. To develop the algorithm, consider a function which can be represented as an undirected chain,

$$f(x_1, x_2, x_3, x_4) = \phi(x_1, x_2) \phi(x_2, x_3) \phi(x_3, x_4) \quad (5.2.2)$$

for which we wish to find the joint state $x_1^*, x_2^*, x_3^*, x_4^*$ which maximises f . Firstly, we calculate the maximum value of f . Since potentials are non-negative, we may write

$$\begin{aligned} \max_{\mathbf{x}} f(\mathbf{x}) &= \max_{x_1, x_2, x_3, x_4} \phi(x_1, x_2) \phi(x_2, x_3) \phi(x_3, x_4) = \max_{x_1, x_2, x_3} \phi(x_1, x_2) \phi(x_2, x_3) \underbrace{\max_{x_4} \phi(x_3, x_4)}_{\gamma_4(x_3)} \\ &= \max_{x_1, x_2} \phi(x_1, x_2) \underbrace{\max_{x_3} \phi(x_2, x_3) \gamma_4(x_3)}_{\gamma_3(x_2)} = \max_{x_1, x_2} \phi(x_1, x_2) \gamma_3(x_2) = \max_{x_1} \underbrace{\max_{x_2} \phi(x_1, x_2) \gamma_3(x_2)}_{\gamma_2(x_1)} \end{aligned}$$

The final equation corresponds to solving a single variable optimisation and determines both the optimal value of the function f and also the optimal state $x_1^* = \operatorname{argmax}_{x_1} \gamma_2(x_1)$. Given x_1^* , the optimal x_2 is given by $x_2^* = \operatorname{argmax}_{x_2} \phi(x_1^*, x_2) \gamma_3(x_2)$, and similarly $x_3^* = \operatorname{argmax}_{x_3} \phi(x_2^*, x_3) \gamma_4(x_3)$, $x_4^* = \operatorname{argmax}_{x_4} \phi(x_3^*, x_4)$. This procedure is called *backtracking*. Note that we could have equally started at the other end of the chain by